1. Software Requirements Specification

CLARA

Document Control	
Author(s)	Michael Gardner
Version	0.1.0
Status	Draft
Status Date	2025-12-29
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	library
Library Role	utility
Application Role	N/A
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

2. Introduction

2.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **CLARA** (Command Line Arguments for Reliable Applications), a type-safe command-line argument parsing library for Ada 2022.

2.2. Scope

CLARA provides:

- Type-safe CLI argument definition via generic instantiation.
- Integration with the Functional library's Result and Option monads.
- Railway-oriented argument processing.
- SPARK-compatible design (no heap allocation).
- Stateless parsing via Ada.Command_Line.

2.3. Definitions and Acronyms

Term	Definition
CLARA	Command Line Arguments for Reliable Applications.
Flag	Boolean switch (e.g., `-v`, `--verbose`).
Generic Instantiation	Ada compile-time polymorphism mechanism.
Option	Switch with a value (e.g., `-o file`, `--output=file`).
Option Monad	Functional pattern for optional values.
Positional	Non-switch arguments (e.g., file paths).
Result Monad	Functional pattern for error handling without exceptions.

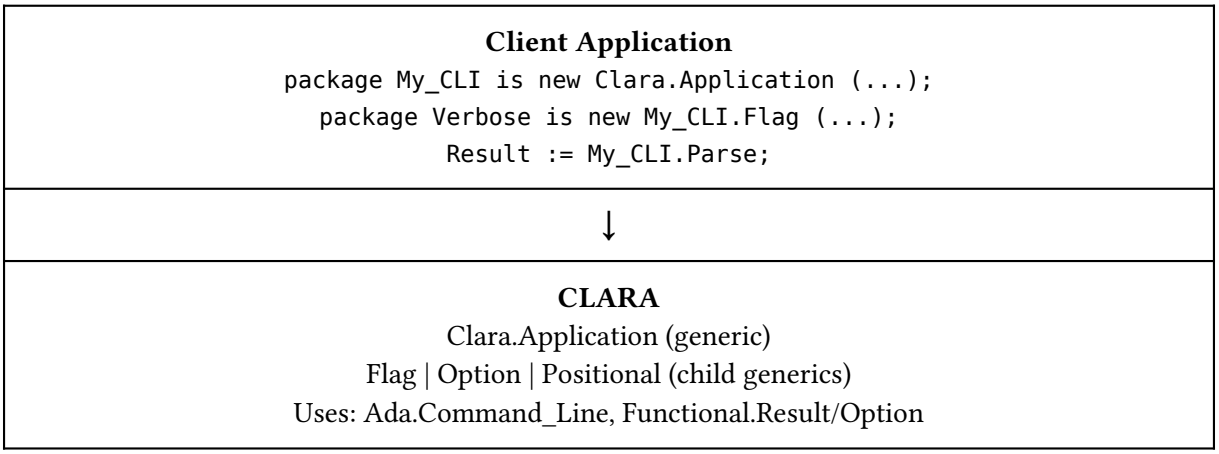
2.4. References

- Ada 2022 Reference Manual (ISO/IEC 8652:2023).
- SPARK 2014 Reference Manual.
- Functional Library — Result/Option monads.

3. Overall Description

3.1. Product Perspective

CLARA is a utility library designed to be imported by Ada applications needing command-line argument parsing. It follows the same generic instantiation pattern as the Functional library.



3.2. Product Features

1. **Flag Definition**: Boolean switches with short/long forms.
2. **Option Definition**: Switches with required/optional values.
3. **Positional Arguments**: File paths and other non-switch arguments.
4. **Type-Safe Access**: Each flag/option is its own package (no string lookups).
5. **Functional Integration**: Parse returns Result; options return Option.
6. **Help Generation**: Automatic help text from definitions.
7. **SPARK Compatible**: No heap allocation, stateless design.

3.3. User Classes

User Class	Description
Application Developers	Ada developers building CLI applications.
Embedded Developers	Developers requiring heap-free, SPARK-compatible patterns.
Library Authors	Developers creating tools with CLI interfaces.

3.4. Operating Environment

Requirement	Specification
Platforms	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Ada Version	Ada 2022
Compiler	GNAT FSF 13+ or GNAT Pro
Dependencies	Functional >= 4.0.0

3.5. Constraints

Constraint	Rationale
Generic Instantiation	Compile-time type safety, consistent with Functional library.
No Heap Allocation	SPARK compatibility and embedded system safety.
No String Lookups	Typos caught at compile time, not runtime.
Stateless Parsing	Avoids GNAT.Command_Line state isolation issues in standalone libraries.

4. Interface Requirements

4.1. User Interfaces

None. This is a library, not an application.

4.2. Software Interfaces

The library exposes a stable Ada API via generic instantiation:

```
with Clara.Application;

package My_CLI is new Clara.Application
  (Name => "myapp", Description => "My application");

package Verbose is new My_CLI.Flag
  (Short => 'v', Long => "verbose", Help => "Enable verbose output");
```

```
package Output is new My_CLI.Option
  (Short => 'o', Long => "output", Value_Name => "FILE",
   Help => "Output file path");
```

4.3. Hardware Interfaces

None. The library is hardware-agnostic.

5. Functional Requirements

5.1. Application Definition (REQ-APP)

Priority: Must **Description:** Provide a generic package for defining CLI applications.

ID	Requirement	Priority
REQ-APP-001	Provide Clara.Application generic for CLI definition.	Must
REQ-APP-002	Application shall have name and description parameters.	Must
REQ-APP-003	Application shall provide Parse function returning Result.	Must
REQ-APP-004	Application shall provide Print_Help procedure.	Must

5.2. Flag Definition (REQ-FLAG)

Priority: Must **Description:** Provide boolean switch arguments.

ID	Requirement	Priority
REQ-FLAG-001	Provide Flag child generic of Application.	Must
REQ-FLAG-002	Flag shall support short form (single character, e.g., `-v`).	Must
REQ-FLAG-003	Flag shall support long form (e.g., `--verbose`).	Must
REQ-FLAG-004	Flag shall support help text parameter.	Must
REQ-FLAG-005	Flag shall provide Is_Set function returning Boolean.	Must
REQ-FLAG-006	Flag shall provide Count function for multiple occurrences.	Should

5.3. Option Definition (REQ-OPT)

Priority: Must **Description:** Provide switches with associated values.

ID	Requirement	Priority
REQ-OPT-001	Provide Option child generic of Application.	Must
REQ-OPT-002	Option shall support short form with value (e.g., `-o file`).	Must
REQ-OPT-003	Option shall support long form with value (e.g., `--output=file`).	Must
REQ-OPT-004	Option shall support value name for help text (e.g., “FILE”).	Must
REQ-OPT-005	Option shall provide Value function returning Option[String].	Must
REQ-OPT-006	Option shall provide Has_Value function returning Boolean.	Must
REQ-OPT-007	Option shall support “or” operator for default values.	Must

5.4. Positional Arguments (REQ-POS)

Priority: Must **Description:** Provide non-switch argument handling.

ID	Requirement	Priority
REQ-POS-001	Provide Positional child generic of Application.	Must

ID	Requirement	Priority
REQ-POS-002	Positional shall support name and help text.	Must
REQ-POS-003	Positional shall support Multiple flag for 0..N values.	Must
REQ-POS-004	Positional shall provide Values function returning a bounded vector.	Must

5.5. Error Handling (REQ-ERR)

Priority: Must **Description:** Provide Result-based parse error reporting.

ID	Requirement	Priority
REQ-ERR-001	Parse shall return Result[Unit, Parse_Error].	Must
REQ-ERR-002	Parse_Error shall include error kind and message.	Must
REQ-ERR-003	Error kinds shall include: Unknown_Switch, Missing_Value, Invalid_Value.	Must
REQ-ERR-004	Errors shall be composable with And_Then and Map_Error.	Must

5.6. Help Generation (REQ-HELP)

Priority: Must **Description:** Provide automatic help text generation from definitions.

ID	Requirement	Priority
REQ-HELP-001	Generate help text from argument definitions.	Must
REQ-HELP-002	Help shall show application name and description.	Must
REQ-HELP-003	Help shall list all flags with short/long forms and help text.	Must
REQ-HELP-004	Help shall list all options with value names.	Must
REQ-HELP-005	Help shall list positional arguments.	Must

5.7. Future: Subcommand Support (REQ-CMD)

Priority: Future **Description:** Support subcommands (e.g., `app cmd --flag`).

ID	Requirement	Priority
REQ-CMD-001	Support subcommands with their own flags and options.	Future
REQ-CMD-002	Application shall route to the appropriate subcommand.	Future

6. Quality and Cross-Cutting Requirements

6.1. Performance (NFR-PERF)

ID	Requirement
NFR-PERF-001	Parsing shall complete in O(n) where n is the argument count.
NFR-PERF-002	No dynamic memory allocation during parsing.

6.2. Portability (NFR-PORT)

ID	Requirement
NFR-PORT-001	The library shall work on POSIX and Windows platforms.
NFR-PORT-002	The library shall use only Ada.Command_Line (standard library).
NFR-PORT-003	No GNAT-specific runtime dependencies.

6.3. SPARK Formal Verification (NFR-SPARK)

ID	Requirement
NFR-SPARK-001	Core packages shall be SPARK_Mode On compatible.
NFR-SPARK-002	No controlled types in core packages.
NFR-SPARK-003	Use bounded data structures only.

Package	SPARK_Mode	Rationale
Clara	On	Root package, pure.
Clara.Application (core)	On	Generic parsing logic.
Clara.Errors	On	Error types, pure.
Clara.Types	On	Core type definitions, bounded strings.
Clara.Version	On	Constants only.

6.4. Maintainability (NFR-MAINT)

ID	Requirement
NFR-MAINT-001	Code coverage shall be at least 90%.
NFR-MAINT-002	All public APIs shall be documented.
NFR-MAINT-003	Design shall be consistent with Functional library patterns.

7. Design and Implementation Constraints

7.1. System Requirements

7.1.1. Hardware Requirements

No specific hardware requirements. The library operates within standard Ada runtime constraints.

7.1.2. Software Requirements

Category	Requirement
Compiler	GNAT FSF 13+ or GNAT Pro (Ada 2022 support).
Package Manager	Alire 2.0+.
Dependencies	Functional >= 4.0.0.
SPARK Prover	GNATprove 14+ (optional, for formal verification).

8. Verification and Traceability

8.1. Verification Methods

Method	Description
Unit Testing	Individual packages: Types, Errors, Flag, Option, Positional.
Integration Testing	Full parsing with multiple argument combinations.
SPARK Proof	Formal verification of core packages.
Code Review	All changes reviewed before merge.

8.2. Traceability Matrix

Requirement	Design	Test
REQ-APP-001	Clara.Application	TC-INT-001..006
REQ-FLAG-001	Clara.Application.Flag	TC-FLAG-001..005
REQ-OPT-001	Clara.Application.Option	TC-OPT-001..006
REQ-POS-001	Clara.Application.Positional	TC-POS-001..005
REQ-ERR-001	Clara.Errors	TC-ERR-001..004
REQ-HELP-001	Clara.Application.Print_Help	TC-HELP-001..004

9. Appendices

9.1. Change History

Version	Date	Author	Changes
0.1.0	2025-12-19	Michael Gardner	Initial requirements from design discussion. Migrate to Typst format.